

L'organisation d'une application React Native

Table des matières

I. Contexte	3
II. Architectures et organisation du code	3
A. Architectures et organisation du code	3
B. Exercice : Quiz	11
III. Outils et pratiques pour maintenir la qualité du code	12
A. Outils et pratiques pour maintenir la qualité du code.....	12
B. Exercice : Quiz	16
IV. Essentiel	17
V. Auto-évaluation	18
A. Exercice	18
B. Test.....	20
Solutions des exercices	21

I. Contexte

Durée : 1 h

Prérequis : bases de programmation

Environnement de travail : ordinateur, Internet, Node.js et npm

Contexte

Dans le paysage dynamique de la technologie, la capacité à créer des applications mobiles robustes et performantes est devenue cruciale pour les entreprises souhaitant offrir une expérience utilisateur optimale. React Native, une bibliothèque développée par Facebook, s'est distinguée comme une solution majeure dans ce domaine, permettant aux développeurs de construire des applications natives pour iOS et Android à partir d'un seul code base en JavaScript.

Toutefois, la puissance de React Native ne réside pas uniquement dans sa portabilité interplateforme, mais aussi dans sa flexibilité. Cette flexibilité, si elle n'est pas maîtrisée, peut cependant conduire à des désordres structurels et à des problèmes de qualité du code. Dans un cadre professionnel, la mise en place d'une architecture propre et bien organisée au sein d'une application React Native est essentielle. Elle garantit non seulement la maintenabilité du code, mais assure également sa scalabilité face à l'évolution des besoins. Imaginez travailler sur un projet d'envergure, avec plusieurs développeurs contribuant simultanément. Sans une architecture claire et des règles strictes, l'application pourrait rapidement devenir un labyrinthe inextricable de code, entravant la productivité et augmentant le risque d'erreurs. D'où l'importance de disposer d'outils pour garantir la qualité du code, tels que les linters, les formateurs et les systèmes d'analyse de qualité.

Ces outils, conjugués à une documentation claire, permettent non seulement de repérer et corriger les problèmes en amont, mais aussi d'assurer une compréhension rapide du code par tous les membres de l'équipe. Enfin, l'utilisation judicieuse de systèmes de gestion de versions comme Git est indispensable. Elle permet de suivre l'évolution du code, de faciliter la collaboration et d'assurer que chaque modification est tracée, commentée et validée.

II. Architectures et organisation du code

A. Architectures et organisation du code

Introduction à l'architecture d'une application React Native

Une architecture propre et organisée est fondamentale dans le développement logiciel pour une multitude de raisons. D'abord et avant tout, elle est essentielle pour garantir la maintenabilité d'une application. Lorsque le code est structuré de manière logique et cohérente, il devient plus facile pour les développeurs, qu'ils soient membres de l'équipe initiale ou nouveaux venus, de comprendre, de modifier et d'ajouter des fonctionnalités à l'application. De plus, une structure claire réduit le risque d'erreurs et d'incohérences dans le code, garantissant ainsi une meilleure qualité et fiabilité du produit final.

Outre ces considérations pragmatiques, une bonne architecture facilite la collaboration entre les membres d'une équipe. Lorsque les règles et structures sont clairement définies, chaque développeur sait où et comment intégrer ses contributions, ce qui minimise les conflits de code et optimise la productivité. Une architecture bien pensée assure également une scalabilité effective, permettant à l'application d'évoluer et de s'adapter aux besoins changeants sans nécessiter de refontes majeures.

En ce qui concerne la comparaison avec les architectures traditionnelles d'applications web, les applications React Native présentent des particularités uniques. Alors qu'une application web traditionnelle repose généralement sur une séparation entre le frontend et le backend, avec éventuellement une architecture MVC (Modèle-Vue-Contrôleur), React Native adopte une approche orientée composant. Dans cette perspective, l'application est conçue comme une collection de composants réutilisables qui encapsulent à la fois la logique et la présentation.

Architecture « feature-based »

L'architecture « *feature-based* » est une approche de structuration du code qui centre l'organisation autour des fonctionnalités ou des caractéristiques spécifiques d'une application plutôt que des types de fichiers ou des couches techniques. Cette méthode est devenue particulièrement populaire dans le monde du développement front-end car elle offre une lisibilité immédiate du code en fonction des besoins métier de l'application. L'essence de cette architecture est de regrouper ensemble tout ce qui concerne une fonctionnalité donnée. Au lieu d'avoir des dossiers séparés pour les modèles, les vues, les contrôleurs ou les services, l'architecture feature-based propose d'avoir un dossier pour chaque fonctionnalité, qui contiendra tous les éléments nécessaires pour cette fonctionnalité spécifique. Cela rend le code plus intuitif, en particulier pour les grandes équipes où plusieurs développeurs peuvent travailler sur différentes fonctionnalités en parallèle.

Exemple

Prenons un exemple pour illustrer cette architecture. Imaginons une application de commerce électronique. Au lieu d'avoir une structure basée sur des dossiers comme « *controllers* », « *models* » et « *views* », vous auriez des dossiers tels que « *Panier* », « *ListeDeProduits* » et « *GestionUtilisateurs* ». Dans le dossier « *Panier* », vous pourriez trouver des fichiers relatifs à l'affichage du panier, à la logique métier du panier, et à la récupération des données du panier. De même, « *ListeDeProduits* » contiendrait tout ce qui est nécessaire pour afficher, filtrer, ou trier une liste de produits :

```

1 /mon-application
2 /src
3   /assets
4     /images
5     /fonts
6     /icons
7
8   /components
9     /Button
10    /Header
11    /Footer
12    ...
13
14  /features
15    /Panier
16      PanierView.js
17      PanierController.js
18      PanierModel.js
19      PanierStyles.css
20
21    /ListeDeProduits
22      ListeDeProduitsView.js
23      ListeDeProduitsController.js
24      ListeDeProduitsModel.js
25      ListeDeProduitsStyles.css
26
27    /GestionUtilisateurs
28      /Login
29        LoginView.js
30        LoginController.js
31        LoginStyles.css
32
33      /SignUp
34        SignUpView.js
35        SignUpController.js
36        SignUpStyles.css
37

```

```
38     ProfileView.js
39     ProfileController.js
40     ProfileStyles.css
41
42     /shared
43     /utils
44     /hooks
45     /api
46     ...
47
48     /app.js
49     /index.js
50
51     /public
52     index.html
53     favicon.ico
54
55     package.json
56     README.md
57     ...
```

Voici une description des dossiers :

- Assets : contient les ressources comme les images, les polices, etc.
- Components : dossiers de composants réutilisables, tels que les boutons, les en-têtes, etc.
- Features : c'est le cœur de l'architecture feature-based. Chaque sous-dossier représente une fonctionnalité distincte. Par exemple, GestionUtilisateurs montre également comment des sous-fonctionnalités, comme Login et SignUp, peuvent être structurées.
- Shared : contient le code partagé, comme les utilitaires, les hooks, les appels d'API, qui sont utilisés à travers différentes fonctionnalités.
- App.js & index.js : points d'entrée principaux de l'application.

Dans le dossier feature, certaines features suivent le pattern suivant : Model-View-Controller (par exemple la liste de produits). Le MVC, ou Modèle-Vue-Contrôleur, est un patron de conception architecturale largement utilisé dans le développement de logiciels pour séparer une application en trois composants interconnectés :

- Le modèle représente les données et la logique métier de l'application. Il se charge de la récupération, du stockage et du traitement des données, par exemple via une base de données ou une API. Le modèle est indépendant des vues et des contrôleurs, ce qui signifie qu'il n'a pas d'informations sur la façon dont les données sont présentées ou sur les actions de l'utilisateur.
- La vue représente l'Interface Utilisateur (UI) de l'application. Elle affiche les données provenant du modèle à l'utilisateur et lui transmet les commandes de l'utilisateur au contrôleur. Elle ne contient pas de logique métier, mais se concentre uniquement sur la présentation.
- Enfin, le contrôleur agit comme un intermédiaire entre le modèle et la vue. Il prend les entrées de l'utilisateur à travers la Vue, les traite (avec éventuellement une mise à jour du modèle) et renvoie la sortie d'affichage à la vue.

Exemple

En utilisant le dossier /ListeDeProduits fourni comme exemple, le fichier ListeDeProduitsModel.js contiendrait la logique métier et les structures de données associées à la liste de produits. Par exemple, il pourrait avoir des fonctions pour ajouter un produit, supprimer un produit, récupérer une liste de produits, etc. ListeDeProduitsView.js se concentrerait sur la façon dont la liste de produits est présentée à l'utilisateur. Il pourrait définir comment chaque produit est affiché, comment la liste est organisée, etc. Enfin,

ListeDeProduitsController.js serait responsable de gérer les interactions entre la vue et le modèle. Par exemple, si un utilisateur décide d'ajouter un nouveau produit via l'interface utilisateur, le contrôleur déciderait de la façon de traiter cette action, en invoquant probablement une méthode du modèle.

Architecture « layer-based »

L'architecture « *layer-based* », également connue sous le nom d'architecture en couches, est une approche classique de structuration du code qui organise les différents éléments d'une application selon leurs responsabilités techniques. Cette méthode repose sur la séparation des préoccupations, où chaque couche du système a une fonction spécifique, permettant ainsi de segmenter le code en modules logiques pour une meilleure gestion et maintenabilité. L'une des principales caractéristiques de l'architecture en couches est qu'une couche donnée ne communique qu'avec la couche immédiatement inférieure ou supérieure. Cela crée une hiérarchie claire qui minimise les dépendances et favorise une évolutivité modulaire.

Exemple

Considérons un exemple simplifié d'une application web. Dans une architecture en couches typique, on pourrait avoir les couches suivantes :

- Une couche de présentation, qui gère tout ce qui est lié à l'interface utilisateur, comme les pages web, les formulaires ou les composants d'interface. Elle se charge de présenter les données à l'utilisateur et de recueillir ses inputs.
- Une couche métier, également connue sous le nom de couche logique ou de contrôleur, contient la logique d'entreprise. C'est ici que se prennent la plupart des décisions, que sont appliquées les règles métier, et que sont traitées les demandes venant de la couche de présentation.
- Une couche d'accès aux données qui interagit directement avec les bases de données ou d'autres sources de données. Elle récupère, modifie et stocke les données sur demande de la couche métier.

Pour illustrer cela, imaginons une application de gestion de bibliothèque. L'utilisateur sélectionne un livre via la couche de présentation (interface web). La demande est ensuite traitée par la couche métier, qui vérifie, par exemple, la disponibilité du livre. Finalement, la couche d'accès aux données interroge la base de données pour obtenir les informations détaillées sur le livre en question.

Voici un exemple :

```

1 /mon-application
2   /src
3     /presentation
4       /components
5         /Button
6         /Header
7         ...
8     /views
9       /HomePage
10      /ProductPage
11      /UserProfile
12      ...
13    /styles
14      global.css
15      ...
16
17    /businessLogic
18      /services
19        UserService.js
20        ProductService.js
21      ...

```

```
22     /controllers
23         UserController.js
24         ProductController.js
25         ...
26     /rules
27         validationRules.js
28         businessRules.js
29         ...
30
31     /dataAccess
32         /models
33             User.js
34             Product.js
35             ...
36         /repositories
37             UserRepository.js
38             ProductRepository.js
39             ...
40         /database
41             connection.js
42             migrations.js
43             ...
44
45     /shared
46         /utils
47         /constants
48         ...
49
50     /app.js
51     /index.js
52
53 /public
54     index.html
55     favicon.ico
56
57 package.json
58 README.md
59 ...
```

Voici une description des dossiers :

- **Presentation** : c'est ici que réside tout ce qui est lié à la couche de présentation. Elle englobe les composants UI, les vues, et les styles associés.
- **BusinessLogic** : cette couche contient la logique métier. Les services représentent des opérations ou des ensembles de fonctionnalités spécifiques (par ex. la gestion des utilisateurs ou des produits), tandis que les contrôleurs orchestrent la logique et la communication entre la présentation et l'accès aux données. Les règles sont des scripts ou des fonctions pour des règles métier spécifiques ou des validations.
- **DataAccess** : c'est la couche qui gère tout ce qui est lié à la communication entre l'application et les sources de données externes. Les endpoints définissent les routes pour accéder aux données, les fonctions d'appel contiennent la logique pour interroger et envoyer des données à ces endpoints, et les configurations peuvent inclure des clés d'API, des paramètres d'authentification et d'autres détails nécessaires pour interagir avec les services externes.

Avantages et inconvénients de chaque structure

Alors que l'architecture feature-based tend à favoriser une organisation centrée sur l'utilisateur ou sur le business, la layer-based se focalise sur la structure technique. Le choix entre elles dépend souvent de la nature du projet, de la taille de l'équipe et des préférences architecturales. Pour un projet axé sur la rapidité de livraison avec des fonctionnalités clairement définies, le feature-based pourrait être préférable. Pour un système complexe nécessitant une séparation claire des responsabilités techniques, le layer-based pourrait être la voie à suivre.

Voici les avantages et inconvénients de l'architecture feature-based :

Avantages	Défauts
Cohésion : tout le code relatif à une fonctionnalité est regroupé, ce qui facilite la compréhension et la maintenance de cette fonctionnalité.	Duplication : il peut y avoir des redondances si plusieurs fonctionnalités nécessitent des logiques similaires.
Évolutivité : ajouter ou retirer des fonctionnalités peut s'avérer plus direct, car cela revient souvent à ajouter ou supprimer des dossiers spécifiques.	Complexité croissante : à mesure que le nombre de fonctionnalités augmente, naviguer entre elles et comprendre les interactions peut devenir complexe.
Développement parallèle : les équipes peuvent travailler sur des fonctionnalités distinctes sans trop d'interférences, car chaque fonctionnalité est relativement isolée.	

Voici les avantages et inconvénients de l'architecture layer-based :

Avantages	Défauts
Séparation des préoccupations : chaque couche a une responsabilité bien définie, ce qui peut faciliter la réutilisation et le remplacement des composants.	Complexité de navigation : lors de la mise à jour d'une fonctionnalité, un développeur pourrait avoir à naviguer entre plusieurs couches, rendant le processus moins intuitif.
Maintenabilité : les modifications dans une couche, comme le changement d'une base de données, ont moins de chances d'affecter les autres couches.	Rigidité : l'introduction de changements qui ne s'adaptent pas bien à cette structure en couches peut s'avérer problématique.

Méthode Gestion de Livres dans une Bibliothèque avec une Architecture Feature-Based

Pour cet exemple, nous allons créer une petite application en React pour illustrer l'architecture feature-based, qui est populaire dans les projets React de par sa modularité et sa clarté. Dans une architecture basée sur les fonctionnalités, les dossiers sont généralement organisés autour de fonctionnalités ou de domaines spécifiques de l'application, plutôt que par type de fichier ou par responsabilité technique :

```

1 /my-library-app
2 |-- /src
3   |-- /features
4     |-- /BookList
5       |-- BookList.js

```

```
6      |-- BookList.css
7      |-- /AddBook
8      |-- AddBook.js
9      |-- AddBook.css
10     |-- App.js
11     |-- index.js
```

Quelques explications :

- /my-library-app : c'est le dossier racine de votre projet.
- /src : Ce dossier contient l'ensemble du code source de votre application.
- /features : un dossier pour regrouper toutes les fonctionnalités. Chaque fonctionnalité ou composant aura son propre sous-dossier.
- /BookList et /AddBook : ce sont des dossiers pour des composants individuels ou des groupes de composants liés à une fonctionnalité spécifique. Vous pouvez également inclure d'autres fichiers associés, comme des tests, des utilitaires ou des styles spécifiques à cette fonctionnalité.
- BookList.js et AddBook.js : ce sont les fichiers pour vos composants fonctionnels. Vous pourriez également avoir des fichiers .css associés pour le style, des fichiers .test.js pour les tests, etc.
- App.js : le composant principal de votre application, qui inclut généralement la navigation et l'agencement global.
- index.js : le point d'entrée de votre application qui monte le composant App sur le DOM.

Si vous démarrez à partir de zéro, vous pouvez utiliser npx pour initialiser un nouveau projet. Cet outil fournit une configuration de base pour commencer rapidement dans la ligne de commande avec :

```
1 npx create-expo-app my-library-app
2 cd my-library-app
```

Un composant fonctionnel est essentiellement une fonction JavaScript qui retourne du JSX (syntaxe similaire à HTML). Pour afficher la liste des livres, vous pouvez utiliser :

```
1 //BookList.js
2 import React from 'react';
3 import './BookList.css';
4
5 const BookList = ({ books }) => (
6   <div className="book-list">
7     <h2>Liste des Livres</h2>
8     <ul>
9       {books.map(book => (
10         <li key={book.id}>{book.title}</li>
11       ))}
12     </ul>
13   </div>
14 );
15
16 export default BookList;
```

`const BookList = ({ books }) => (` déclare une constante nommée `BookList` qui est une fonction. Cette fonction est un exemple de composant fonctionnel en React. La syntaxe `{ books }` est une déstructuration des props, ce qui signifie que nous nous attendons à ce que ce composant reçoive une prop nommée `books`. À l'intérieur de ce composant, nous avons du JSX, qui ressemble beaucoup à du HTML. Le JSX permet aux développeurs de définir l'interface utilisateur de manière déclarative.

Dans ce JSX, nous avons une `div` qui contient un en-tête (`h2`) et une liste non ordonnée (`ul`). Cette liste est générée dynamiquement à partir de la prop `books`.

La ligne `{books.map(book => (` utilise la méthode `map` de JavaScript pour itérer sur chaque livre dans la liste `books`. Pour chaque livre, elle crée un élément de liste (`li`) avec le titre du livre. La propriété `key` dans `<li key={book.id}>` est un élément unique que React utilise pour optimiser le rendu des listes. Elle aide React à savoir quel élément a changé, est ajouté ou est supprimé. Pour finir, `export default BookList;` exporte le composant pour qu'il puisse être utilisé ailleurs dans l'application.

Voici le CSS associé :

```
1 .book-list {
2   font-family : 'Arial', sans-serif;
3 }
```

On définit le composant `AddBook` qui affiche un formulaire pour ajouter un livre dans la liste :

```
1 //AddBook.js
2 import React, { useState } from 'react';
3 import './AddBook.css';
4
5 const AddBook = ({ onAdd }) => {
6   const [title, setTitle] = useState('');
7
8   const handleSubmit = (e) => {
9     e.preventDefault();
10    if(title) {
11      onAdd(title);
12      setTitle('');
13    }
14  };
15
16  return (
17    <div className="add-book">
18      <h2>Ajouter un Livre</h2>
19      <form onSubmit={handleSubmit}>
20        <input
21          type="text"
22          placeholder="Titre du livre"
23          value={title}
24          onChange={(e) => setTitle(e.target.value)}
25        />
26        <button type="submit">Ajouter</button>
27      </form>
28    </div>
29  );
30 }
31
32 export default AddBook;
```

Le code importe React et le Hook `useState`, permettant au composant fonctionnel d'accéder aux fonctionnalités de React, telles que le state. Le Hook `useState` est employé pour gérer le titre du livre que l'utilisateur saisit. Les styles spécifiques au composant sont intégrés grâce à l'instruction `import './AddBook.css';`. Le composant lui-même est déclaré comme `const AddBook`, et grâce à la déstructuration, la prop `onAdd` est directement accessible. Cette prop est une fonction appelée lors de l'ajout d'un nouveau livre.

À l'intérieur du composant, une variable d'état `title` est créée avec `useState`, initialement définie comme une chaîne vide. La fonction `handleSubmit` est ensuite définie pour traiter la soumission du formulaire. Elle vérifie si un titre a été saisi avant de déclencher la fonction `onAdd`. Le rendu JSX décrit la structure visuelle du composant, incluant un champ de saisie lié à la variable d'état `title`, qui se met à jour à chaque modification. L'action du bouton « *Ajouter* » provoque la soumission du formulaire et donc l'appel de `handleSubmit`. Enfin, le composant est exporté pour une utilisation dans d'autres fichiers.

Voici le CSS associé :

```
1 .add-book {  
2   font-family : 'Arial', sans-serif;  
3 }
```

Vous pouvez ensuite intégrer ce composant dans votre App.js ou n'importe quel autre composant parent.

B. Exercice : Quiz

[solution n°1 p.23]

Question 1

Quelle est l'idée principale derrière l'architecture feature-based ?

- ☐ Organiser le code selon les types de fichiers
- ☐ Structurer le code en fonction des couches techniques
- ☐ Centrer l'organisation du code autour des fonctionnalités spécifiques d'une application

Question 2

Quel dossier contient des composants qui peuvent être réutilisés dans différentes parties de l'application ?

- ☐ /assets
- ☐ /features
- ☐ /components

Question 3

Quel est l'objectif principal de l'architecture « *layer-based* » ou architecture en couches ?

- ☐ Segmenter le code en modules logiques selon leurs responsabilités techniques
- ☐ Organiser le code selon les fonctionnalités de l'application
- ☐ Structurer le code en fonction de la hiérarchie des développeurs

Question 4

Comment une couche de l'architecture en couches communique-t-elle généralement ?

- ☐ Elle communique avec toutes les autres couches indépendamment
- ☐ Elle communique uniquement avec la couche immédiatement inférieure ou supérieure
- ☐ Elle communique *via* des messages cryptés aux autres couches

Question 5

Quelle architecture pourrait être préférable pour un projet axé sur la rapidité de livraison avec des fonctionnalités clairement définies ?

- ☐ Architecture layer-based
- ☐ Architecture feature-based
- ☐ Architecture centrée sur la base de données

III. Outils et pratiques pour maintenir la qualité du code

A. Outils et pratiques pour maintenir la qualité du code

Garantir la propreté du code avec les linters et les formatters

L'utilisation d'outils de linting comme ESLint est essentielle pour assurer la qualité du code dans les projets de développement, et cela est particulièrement vrai pour les applications React Native. ESLint aide les développeurs à identifier et corriger les problèmes dans leur code avant l'exécution, garantissant ainsi une base de code propre, performante et exempte d'erreurs courantes. Intégrer ESLint dans un projet React Native commence par son installation et sa configuration.

Méthode	Installer ESLint
	Si vous n'avez pas encore configuré ESLint pour votre projet, commencez par l'installer : <pre>1 npm install eslint --save-dev</pre> Ensuite, initialisez la configuration d'ESLint : <pre>1 npx eslint -init</pre> Lors de l'initialisation, ESLint vous posera quelques questions concernant votre environnement et vos préférences de style de code pour vous aider à créer un fichier de configuration de base. Pour ajouter des règles spécifiques à React Native, vous voudrez utiliser le plugin eslint-plugin-react-native : <pre>1 npm install eslint-plugin-react-native --save-dev</pre> Après avoir installé le plugin, vous devrez l'ajouter à votre configuration ESLint. Ouvrez votre fichier .eslintrc (ou .eslintrc.js ou .eslintrc.json, selon la configuration que vous avez choisie) et modifiez les règles souhaitées : <pre>1 { 2 "extends" : ["eslint:recommended"], 3 "plugins" : ["react-native"], 4 "parserOptions" : { 5 "ecmaFeatures" : { 6 "jsx": true 7 } 8 }, 9 "env" : { 10 "react-native/react-native" : true 11 }, 12 "rules" : { 13 "react-native/no-unused-styles" : 2, 14 "react-native/split-platform-components" : 2, 15 "react-native/no-inline-styles" : 2, 16 "react-native/no-color-literals" : 2, 17 "react-native/no-raw-text" : 2 18 // ... ajoutez d'autres règles comme vous le souhaitez 19 } 20 }</pre> Décortiquons chaque partie de ce fichier : - extends : cette section étend un ensemble prédéfini de règles recommandées par ESLint. "eslint:recommended" : cela ajoute un ensemble de règles qui sont considérées comme des bonnes pratiques par l'équipe ESLint. - plugins : il s'agit d'une liste des plugins que vous souhaitez utiliser. "react-native" : cela indique que le plugin eslint-plugin-react-native sera utilisé, ce qui ajoute des règles spécifiques à React Native.

- `parserOptions` : ceci définit les options pour le parseur que ESLint utilisera pour analyser votre code.
`"jsx" : true` : autorise le linting du code JSX, qui est utilisé par React et React Native.
- `rules` : ici, vous définissez vos propres règles ou modifiez celles que vous avez étendues.
 - `"react-native/no-unused-styles" : 2` : prévient si des styles sont définis mais jamais utilisés.
 - `"react-native/split-platform-components" : 2` : oblige à scinder les composants spécifiques à une plate-forme (comme `.android.js` ou `.ios.js`).
 - `"react-native/no-inline-styles" : 2` : décourage l'utilisation de styles en ligne, car cela peut affecter les performances et est généralement considéré comme une mauvaise pratique en React Native.
 - `"react-native/no-color-literals" : 2` : prévient si des couleurs sont utilisées directement dans les styles plutôt que d'utiliser une variable.
 - `"react-native/no-raw-text" : 2` : s'assure que le texte est toujours dans un composant Text en React Native.

Les numéros 2 à côté de chaque règle signifient que cette règle est « *activée* » et qu'une violation entraînera une erreur. Dans ESLint, 0 signifie « *off* » (désactivé), 1 signifie « *warn* » (avertissement) et 2 signifie « *error* » (erreur).

Il existe beaucoup d'autres règles configurables, par exemple, des règles qui empêchent l'utilisation de `console.log` pour éviter les logs inutiles dans une application en production ou des règles qui garantissent une bonne structure de composants pour optimiser les performances.

Utilisation de Prettier pour le formatage du code

Prettier est un outil de formatage de code automatique, prisé pour sa capacité à améliorer la lisibilité du code en standardisant sa présentation. L'un des avantages majeurs de Prettier est qu'il prend le code existant et le reformate selon un ensemble de règles de style prédéfinies, éliminant ainsi les débats au sein de l'équipe sur le style du code. Cette approche « *opinionnée* » permet de se concentrer sur la logique du code plutôt que sur son apparence, favorisant ainsi la productivité. Contrairement aux linters traditionnels, qui se concentrent principalement sur la manière dont le code est écrit du point de vue de la syntaxe et de la logique, Prettier se focalise exclusivement sur la manière dont le code est présenté. Il se débarrasse des configurations détaillées et des choix minutieux sur les styles. Au lieu de cela, avec quelques options bien choisies, Prettier assure une cohérence dans l'ensemble du projet.

Cela nous amène à la distinction entre Prettier et ESLint. ESLint est un outil de linting qui identifie et rapporte les modèles trouvés dans le code ECMAScript/JavaScript, le tout en se basant sur les règles définies. Ces règles peuvent concerner à la fois des erreurs potentielles dans le code ou des conventions de style. Bien qu'ESLint puisse également corriger certains problèmes automatiquement, son objectif principal est d'identifier des erreurs ou des pratiques qui pourraient conduire à des bugs. En revanche, Prettier ignore totalement ces aspects logiques et se concentre uniquement sur la présentation visuelle du code.

Dans l'écosystème moderne du développement, Prettier et ESLint sont souvent utilisés conjointement. Prettier formate le code pour qu'il soit propre et cohérent, tandis qu'ESLint s'assure que le code est exempt d'erreurs potentielles. L'utilisation combinée de ces outils permet d'obtenir un code non seulement esthétiquement plaisant, mais aussi fonctionnellement robuste.

Méthode Installer Prettier

Installer et configurer Prettier est un processus similaire à ESLint (et à beaucoup d'autres outils). Installez Prettier en tant que dépendance de développement dans votre projet avec la commande suivante :

```
1 npm install prettier --save-dev
```

Pour configurer Prettier, créez un fichier `.prettierrc` à la racine de votre projet. Dans ce fichier, vous pouvez spécifier vos préférences de formatage. Par exemple :

```
1 {
2   "singleQuote" : true,
3   "trailingComma": "all",
4   "bracketSpacing": true,
5   "tabWidth": 2,
6   "semi": true
7 }
```

Les options ci-dessus font en sorte que Prettier :

- Utilise des guillemets simples au lieu des doubles,
- Ajoute une virgule à la fin lors de l'énumération d'objets ou d'arrays,
- Ajoute un espace entre les accolades,
- Utilise une indentation de 2 espaces,
- Termine toujours les déclarations par un point-virgule.

Pour faciliter l'utilisation de Prettier, ajoutez des scripts dans votre fichier `package.json` :

```
1 "scripts" : {
2   "format" : "prettier --write \"src/**/*.{js,jsx,ts,tsx,json,css,md}\"",
3   "format-check" : "prettier --check \"src/**/*.{js,jsx,ts,tsx,json,css,md}\""
4 }
```

Vous pourrez lancer les commandes :

- `"npm run format"` qui formate automatiquement les fichiers,
- `"npm run format-check"` qui vérifie qu'ils sont bien formatés, sans les modifier.

Si vous utilisez également ESLint, il peut être judicieux d'intégrer Prettier pour éviter les conflits de style.

Installez `eslint-plugin-prettier` et `eslint-config-prettier` :

```
1 npm install --save-dev eslint-plugin-prettier eslint-config-prettier
```

Puis, mettez à jour votre configuration ESLint pour inclure ces extensions :

```
1 {
2   "extends" : ["plugin:prettier/recommended"],
3   "plugins" : ["prettier"],
4   "rules" : {
5     "prettier/prettier" : "error"
6   }
7 }
```

Avec ces étapes, vous aurez configuré Prettier pour qu'il travaille harmonieusement avec votre projet, assurant ainsi un formatage de code cohérent à chaque fois.

Outils de qualité de code : SonarLint

SonarLint est un outil de pointe destiné à améliorer la qualité du code en identifiant et en suggérant des corrections pour les défauts de code en temps réel. Il s'intègre Directement dans les Environnements de Développement (IDE) et est conçu pour fonctionner de manière transparente avec de nombreux langages, y compris JavaScript.

Lorsqu'il s'agit de développer des applications en React Native, SonarLint se présente comme un atout précieux pour les développeurs, car il assure une vérification continue de la qualité du code pendant que vous écrivez. Au cœur de SonarLint se trouve un moteur d'analyse statique qui évalue le code source à la recherche de problèmes potentiels. Contrairement à d'autres outils qui fonctionnent en post-traitement ou lors des phases de construction, SonarLint opère en temps réel, fournissant des feedbacks immédiats aux développeurs pendant qu'ils codent. Cela

signifie qu'il est non seulement capable d'identifier des erreurs, mais également de suggérer des corrections, des améliorations et de fournir des explications pour chaque anomalie détectée. L'objectif est d'éduquer le développeur, lui permettant de comprendre la nature du problème et de l'éviter à l'avenir.

L'intégration de SonarLint avec React Native est relativement simple. Premièrement, SonarLint doit être installé comme une extension ou un plugin de votre IDE préféré. La plupart des IDE populaires, tels que Visual Studio Code, IntelliJ IDEA et Eclipse, sont compatibles avec SonarLint. Chaque IDE a sa méthode d'installation. Une fois installé, il commence automatiquement à analyser le code JavaScript et à fournir des feedbacks.

Pour les projets React Native, certaines règles spécifiques peuvent être plus pertinentes que d'autres. Heureusement, SonarLint offre une configuration flexible, permettant aux développeurs de personnaliser les règles selon les besoins du projet. Ainsi, vous pouvez choisir de n'activer que les règles qui sont pertinentes pour votre application React Native.

Il est important de noter que, bien que SonarLint ait des similitudes avec des outils comme ESLint ou Prettier, son rôle est différent. Tandis qu'ESLint et Prettier se concentrent principalement sur le style et la conformité du code, SonarLint va au-delà en se concentrant sur la qualité du code, identifiant des problèmes qui peuvent avoir un impact sur la performance, la sécurité et la maintenabilité.

Exemple

Voici un exemple simplifié de la manière dont SonarLint fonctionne. Supposons que vous travailliez sur une fonctionnalité de votre application web où vous avez besoin de récupérer des données d'une API. Vous écrivez le code suivant dans votre éditeur :

```
1 function fetchData(url) {  
2   let response = fetch(url);  
3   let data = response.json();  
4   return data;  
5 }
```

Dès que vous avez terminé d'écrire ces lignes, SonarLint, intégré à votre IDE, commence immédiatement son analyse en arrière-plan. Une fois l'analyse terminée, vous pourriez voir une ligne ondulée sous `let response = fetch(url);`. En plaçant votre curseur sur cette ligne, SonarLint affiche un message d'avertissement tel que : « *Missing 'await' for an async function call.* » L'appel à la fonction `fetch` est asynchrone. Sans l'utilisation du mot-clé `await`, la variable `response` ne contiendra pas la réponse de la requête, mais une promesse non résolue. Cela pourrait entraîner des erreurs si vous essayez d'appeler `response.json()` sur cette promesse non résolue. SonarLint suggérera une solution pour ce problème, par exemple :

```
1 async function fetchData(url) {  
2   let response = await fetch(url);  
3   let data = await response.json();  
4   return data;  
5 }
```

Documentation avec JSDoc

La documentation du code est un élément vital de tout projet logiciel. À première vue, le codage peut sembler être un acte de pure ingénierie, une tâche technique visant à résoudre un problème donné. Cependant, il convient de reconnaître que le code est aussi une forme de communication. C'est la façon dont les développeurs parlent non seulement à la machine, mais aussi, et peut-être surtout, à d'autres développeurs, y compris leur futur soi. L'un des principaux avantages de la documentation est qu'elle facilite la maintenance et l'évolutivité du logiciel. Un code bien documenté réduit le temps nécessaire pour comprendre le fonctionnement d'un système, permettant ainsi aux développeurs de modifier, corriger ou améliorer le code plus rapidement. En outre, lorsque des équipes changent ou que de nouveaux membres rejoignent un projet, la documentation du code sert de guide, réduisant ainsi la courbe d'apprentissage.

Au milieu de cette prise de conscience de l'importance de la documentation, JSDoc émerge comme un outil inestimable pour le monde JavaScript. JSDoc est un outil de documentation pour JavaScript, qui lit les commentaires dans le code source et génère une documentation complète. Son format est basé sur des commentaires spécialement formatés, qui permettent aux développeurs de décrire leurs fonctions, classes, composants et autres structures de manière claire et structurée.

Lorsqu'il s'agit de documenter des composants, des fonctions ou des classes, il y a des directives essentielles à suivre. Pour les fonctions, il est crucial d'expliquer l'objectif de la fonction, ses paramètres, les types attendus, la valeur de retour et toute exception qu'elle pourrait lancer.

Exemple

Par exemple, avec JSDoc, un commentaire pourrait ressembler à :

```
1 /**
2  * Calcule la somme de deux nombres.
3  * @param {number} a - Premier nombre.
4  * @param {number} b - Deuxième nombre.
5  * @returns {number} - La somme de a et b.
6  */
7 function somme(a, b) {
8     return a + b;
9 }
10 /** @type {number} */
11 let maVariable;
12 /**
13  * @class
14  */
15 function MaClasse() { /* ... */ }
```

JSDoc possède plusieurs mots-clés, appelés « tags », qui sont préfixés par un @. Ces tags aident à fournir des informations spécifiques sur divers éléments du code. Voici une liste des mots-clés importants de JSDoc, avec une brève description de leur utilisation : le mot-clé @param décrit le paramètre de la fonction. Le mot @returns décrit ce que renvoie la fonction. @class (ou @constructor) indique qu'une fonction est un constructeur. @type indique un type pour une variable, un paramètre ou une valeur renvoyée. @deprecated indique que l'utilisation de cette partie du code est déconseillée et beaucoup d'autres.

Méthode Génération de la documentation HTML

Gestion des versions avec Git

B. Exercice : Quiz

[solution n°2 p.24]

Question 1

Quel est le principal objectif de l'utilisation d'outils de linting comme ESLint dans les projets de développement ?

- ☐ Minimiser le temps de chargement de l'application
- ☐ Assurer la qualité du code en identifiant et corrigeant les problèmes avant l'exécution
- ☐ Augmenter la compatibilité du code avec différents navigateurs

Question 2

Quelle est la signification du numéro 2 à côté de chaque règle dans ESLint ?

- ☐ La règle est désactivée
- ☐ La règle renvoie un avertissement lorsqu'elle est violée
- ☐ La règle renvoie une erreur lorsqu'elle est violée

Question 3

Quel est le principal objectif de Prettier ?

- ☐ Identifier les erreurs potentielles dans le code
- ☐ Optimiser le code pour améliorer les performances
- ☐ Améliorer la lisibilité du code en standardisant sa présentation

Question 4

Quelle est la principale distinction entre Prettier et ESLint ?

- ☐ Prettier se concentre sur le formatage visuel du code, tandis qu'ESLint identifie des erreurs ou des pratiques qui pourraient conduire à des bugs
- ☐ Prettier se concentre sur les erreurs logiques, tandis qu'ESLint se concentre sur le formatage
- ☐ Prettier et ESLint ont exactement les mêmes objectifs

Question 5

Quelle est la principale utilité de JSDoc dans le développement JavaScript ?

- ☐ Détecter et corriger les bugs automatiquement
- ☐ Transformer le code JavaScript en code Java
- ☐ Lire les commentaires dans le code source et générer une documentation complète

IV. Essentiel

L'architecture « *feature-based* » est une approche structurale qui organise le code selon les fonctionnalités ou caractéristiques spécifiques d'une application, ce qui facilite la localisation et la gestion des fonctionnalités individuelles. En contraste, l'architecture « *layer-based* » segmente le code en couches ou niveaux logiques, tels que la présentation, la logique métier et l'accès aux données, garantissant une séparation claire des préoccupations à travers le système.

Dans le cadre de l'assurance qualité du code, ESLint est un outil d'analyse statique largement utilisé pour identifier et signaler les motifs problématiques trouvés dans le code ECMAScript, favorisant ainsi une base de code cohérente et sans bugs. En complément, Prettier est un formateur de code qui assure que le code est non seulement fonctionnel, mais aussi bien formaté et lisible, simplifiant le processus de révision.

De même, SonarLint est un autre outil d'analyse statique conçu pour détecter les erreurs, bugs et vulnérabilités dans le code source, offrant ainsi une autre couche de vérification. JSDoc, de son côté, est un outil essentiel pour la documentation, qui lit les commentaires formatés dans le code source JavaScript et génère une documentation complète à partir de ceux-ci.

Enfin, Git se présente comme un système de contrôle de version distribué, permettant aux équipes de suivre et gérer les modifications apportées au code source, facilitant la collaboration et la gestion des versions.

V. Auto-évaluation

A. Exercice

Vous êtes ingénieur logiciel chez « *FutureTech Navigations* », une entreprise avant-gardiste déterminée à redéfinir la manière dont nous interagissons avec nos environnements urbains. Votre tâche principale est de concevoir une application pour lunettes de réalité augmentée qui guide les utilisateurs à travers les rues labyrinthiques des métropoles modernes.

Question 1

[solution n°3 p.25]

Votre plateforme propose plusieurs fonctionnalités telles que la recherche d'itinéraires, la sauvegarde de lieux favoris, les commentaires et avis des utilisateurs sur des lieux spécifiques. Dressez une liste des caractéristiques que vous pensez importantes pour chaque fonctionnalité. Comment organiseriez-vous votre projet en dossiers et fichiers selon une architecture basée sur les fonctionnalités ?

Question 2

[solution n°4 p.25]

Voici le code du composant MapView.js (fictif), chargé d'afficher une carte interactive. Commentez ce fichier avec JsDoc :

```

1 import React from 'react';
2
3 class MapView extends React.Component {
4
5   constructor(props) {
6     super(props);
7     this.state = {
8       center: {
9         lat: 0,
10        lng: 0
11      },
12      zoom : 12,
13      routes : [],
14      pointsOfInterest: [],
15    };
16
17    this.mapRef = React.createRef();
18  }
19
20  componentDidMount() {
21    this.initializeMap();
22    this.fetchRoutes();
23    this.fetchPointsOfInterest();
24  }
25
26  initializeMap() {
27    // Simulons l'initialisation d'une carte avec une bibliothèque fictive
28    this.map = new FakeMapLibrary(this.mapRef.current, {
29      center: this.state.center,
30      zoom : this.state.zoom,
31    });
32  }
33
34  fetchRoutes() {
35    // Simulons une requête pour obtenir des routes
36    fakeAPI.getRoutes().then(routes => {
37      this.setState({ routes });
38    });
39  }
40

```

```

41  fetchPointsOfInterest() {
42      // Simulons une requête pour obtenir des points d'intérêt
43      fakeAPI.getPOIs().then(pointsOfInterest => {
44          this.setState({ pointsOfInterest });
45      });
46  }
47
48  handleZoomChange(zoom) {
49      this.setState({ zoom });
50  }
51
52  handleCenterChange(lat, lng) {
53      this.setState({
54          center: { lat, lng }
55      });
56  }
57
58  renderRoutes() {
59      return this.state.routes.map(route => (
60          <FakeRouteComponent key={route.id} data={route} />
61      ));
62  }
63
64  renderPointsOfInterest() {
65      return this.state.pointsOfInterest.map(poi => (
66          <FakePOIComponent key={poi.id} data={poi} />
67      ));
68  }
69
70  render() {
71      return (
72          <div ref={this.mapRef} className="map-view">
73              {this.renderRoutes()}
74              {this.renderPointsOfInterest()}
75          </div>
76      );
77  }
78 }
79
80 const fakeAPI = {
81     getRoutes: () => Promise.resolve([ { id: 1 }, { id: 2 } ]),
82     getPOIs: () => Promise.resolve([ { id: 'a' }, { id: 'b' } ]),
83 };
84
85 class FakeMapLibrary {
86     constructor(element, options) {
87         // Simulons une initialisation de carte
88     }
89 }
90
91 class FakeRouteComponent extends React.Component {
92     render() {
93         // Affiche une route fictive
94         return <div className="fake-route"></div>;
95     }
96 }
97
98 class FakePOIComponent extends React.Component {

```

```

99   render() {
100     // Affiche un point d'intérêt fictif
101     return <div className="fake-poi"></div>;
102   }
103 }
104
105 export default MapView;

```

B. Test

Exercice 1 : Quiz

[solution n°5 p.29]

Question 1

Pourquoi l'architecture feature-based est-elle considérée comme bénéfique pour les grandes équipes ?

- ☐ Elle propose une structure unique qui doit être suivie strictement
- ☐ Elle offre une lisibilité immédiate du code selon les besoins métier
- ☐ Elle facilite la création de couches techniques distinctes

Question 2

Quelle couche dans une architecture layer-based est principalement responsable de la logique d'entreprise et du traitement des demandes venant de la couche de présentation ?

- ☐ Couche d'accès aux données
- ☐ Couche de présentation
- ☐ Couche métier

Question 3

Après avoir installé le plugin eslint-plugin-react-native, où devriez-vous l'ajouter pour le configurer ?

- ☐ Dans un fichier appelé .eslintconfig
- ☐ Dans le fichier package.json
- ☐ Dans le fichier .eslintrc ou ses variantes (.eslintrc.js, .eslintrc.json)

Question 4

Quelle règle ESLint vous avertirait si des styles sont définis, mais jamais utilisés dans un projet React Native ?

- ☐ "react-native/no-unused-styles"
- ☐ "react-native/no-raw-text"
- ☐ "react-native/no-color-literals"

Question 5

Quelle information le mot-clé @param de JSDoc décrit-il ?

- ☐ L'objectif général de la fonction
- ☐ Le paramètre d'une fonction
- ☐ La valeur renvoyée par la fonction

Question 6

Comment pouvez-vous générer une documentation HTML à partir des commentaires JSDoc dans vos fichiers JavaScript ?

- ☐ En utilisant la commande `npm install jsdoc`
- ☐ En ouvrant les fichiers dans un navigateur
- ☐ En exécutant la commande `jsdoc .` à la racine du projet

Question 7

Après avoir exécuté la commande pour générer la documentation avec JSDoc, où est généralement stockée la documentation HTML ?

- ☐ Dans un dossier nommé `out`
- ☐ Dans un fichier `documentation.html`
- ☐ Dans un dossier nommé `src`

Solutions des exercices

Exercice p. 11 Solution n°1**Question 1**

Quelle est l'idée principale derrière l'architecture feature-based ?

- ☐ Organiser le code selon les types de fichiers
- ☐ Structurer le code en fonction des couches techniques
- ☒ Centrer l'organisation du code autour des fonctionnalités spécifiques d'une application

 L'architecture « *feature-based* » vise à regrouper ensemble tout ce qui concerne une fonctionnalité donnée. Au lieu de séparer les modèles, vues et contrôleurs dans des dossiers distincts, on a un dossier pour chaque fonctionnalité qui contient tous les éléments nécessaires pour cette fonction.

Question 2

Quel dossier contient des composants qui peuvent être réutilisés dans différentes parties de l'application ?

- ☐ /assets
- ☐ /features
- ☒ /components

 Le dossier « *components* » est conçu pour contenir des composants réutilisables tels que les boutons, les entêtes, etc. Ces composants peuvent être utilisés dans différentes fonctionnalités ou parties de l'application.

Question 3

Quel est l'objectif principal de l'architecture « *layer-based* » ou architecture en couches ?

- ☒ Segmenter le code en modules logiques selon leurs responsabilités techniques
- ☐ Organiser le code selon les fonctionnalités de l'application
- ☐ Structurer le code en fonction de la hiérarchie des développeurs

 L'architecture « *layer-based* » vise à structurer le code selon les responsabilités techniques, chaque couche ayant une fonction spécifique, pour une meilleure gestion et maintenabilité du système.

Question 4

Comment une couche de l'architecture en couches communique-t-elle généralement ?

- ☐ Elle communique avec toutes les autres couches indépendamment
- ☒ Elle communique uniquement avec la couche immédiatement inférieure ou supérieure
- ☐ Elle communique *via* des messages cryptés aux autres couches

 L'une des principales caractéristiques de l'architecture en couches est qu'une couche donnée ne communique qu'avec la couche immédiatement au-dessus ou en dessous d'elle, ce qui crée une hiérarchie claire.

Question 5

Quelle architecture pourrait être préférable pour un projet axé sur la rapidité de livraison avec des fonctionnalités clairement définies ?

- ☐ Architecture layer-based
- ☒ Architecture feature-based
- ☐ Architecture centrée sur la base de données
-  L'architecture feature-based tend à favoriser une organisation centrée sur l'utilisateur ou sur le business, ce qui pourrait être préférable pour un projet axé sur la rapidité de livraison avec des fonctionnalités clairement définies.

Exercice p. 16 Solution n°2

Question 1

Quel est le principal objectif de l'utilisation d'outils de linting comme ESLint dans les projets de développement ?

- ☐ Minimiser le temps de chargement de l'application
- ☒ Assurer la qualité du code en identifiant et corrigeant les problèmes avant l'exécution
- ☐ Augmenter la compatibilité du code avec différents navigateurs
-  ESLint aide les développeurs à identifier et corriger les problèmes dans leur code avant son exécution, garantissant une base de code propre, performante et exempte d'erreurs courantes.

Question 2

Quelle est la signification du numéro 2 à côté de chaque règle dans ESLint ?

- ☐ La règle est désactivée
- ☐ La règle renvoie un avertissement lorsqu'elle est violée
- ☒ La règle renvoie une erreur lorsqu'elle est violée
-  Dans ESLint, le numéro 2 à côté d'une règle signifie que cette règle est « *activée* » et qu'une violation entraînera une erreur.

Question 3

Quel est le principal objectif de Prettier ?

- ☐ Identifier les erreurs potentielles dans le code
- ☐ Optimiser le code pour améliorer les performances
- ☒ Améliorer la lisibilité du code en standardisant sa présentation
-  Prettier est un outil de formatage de code qui se concentre sur la manière dont le code est présenté visuellement.

Question 4

Quelle est la principale distinction entre Prettier et ESLint ?

- ☒ Prettier se concentre sur le formatage visuel du code, tandis qu'ESLint identifie des erreurs ou des pratiques qui pourraient conduire à des bugs
- ☐ Prettier se concentre sur les erreurs logiques, tandis qu'ESLint se concentre sur le formatage
- ☐ Prettier et ESLint ont exactement les mêmes objectifs

- 🔍 Prettier est axé sur la présentation visuelle du code, tandis qu'ESLint se focalise sur la syntaxe et la logique du code.

Question 5

Quelle est la principale utilité de JSDoc dans le développement JavaScript ?

- ☐ Détecter et corriger les bugs automatiquement
 - ☐ Transformer le code JavaScript en code Java
 - ☒ Lire les commentaires dans le code source et générer une documentation complète
- 🔍 JSDoc est conçu pour lire des commentaires spécialement formatés dans le code source JavaScript et générer une documentation à partir de ces commentaires.

p. 18 Solution n°3

Il n'y a pas qu'une seule solution valide, mais on peut utiliser celle-ci :

- Pour la recherche d'itinéraires : calcul d'itinéraire optimal, points d'intérêt le long de l'itinéraire, options d'itinéraire à pied, en voiture, ou à vélo.
- Pour la sauvegarde de lieux favoris : option de marquage d'un lieu comme favori, gestion des lieux favoris (ajout, suppression, édition).
- Commentaires et avis des utilisateurs sur des lieux spécifiques : ajout de commentaires et de notes.

Organisation des dossiers et fichiers selon une architecture basée sur les fonctionnalités :

```

1 FutureTechNavigations/
2 |
3 |— RechercheItineraires/
4 |   |— index.js
5 |   |— service.js
6 |   |— components/
7 |       |— MapView.js
8 |       |— TrafficOverlay.js
9 |       |— RouteOptions.js
10 |
11 |— LieuxFavoris/
12 |   |— index.js
13 |   |— service.js
14 |   |— components/
15 |       |— FavoritesList.js
16 |       |— AddFavorite.js
17 |       |— EditFavorite.js
18 |— CommentairesAvis/
19 |   |— index.js
20 |   |— service.js
21 |   |— components/
22 |       |— ReviewList.js
23 |       |— AddReview.js
24 |       |— ReportReview.js
  
```

p. 18 Solution n°4

Les commentaires JSDoc sont placés au-dessus des méthodes, des classes et des fonctions pour décrire leur but et leurs paramètres. Cela aidera les autres développeurs à comprendre rapidement le code et à savoir comment l'utiliser.

```

1 import React from 'react';
2
3 /**
4  * Represents a map view component.
5  * @extends React.Component
6  */
7 class MapView extends React.Component {
8
9     /**
10     * Constructs the MapView component.
11     * @param {Object} props - The properties passed to the component.
12     */
13     constructor(props) {
14         super(props);
15         this.state = {
16             center: {
17                 lat: 0,
18                 lng: 0
19             },
20             zoom : 12,
21             routes : [],
22             pointsOfInterest: [],
23         };
24
25         this.mapRef = React.createRef();
26     }
27
28     /**
29     * Initializes the map and fetches the routes and points of interest.
30     */
31     componentDidMount() {
32         this.initializeMap();
33         this.fetchRoutes();
34         this.fetchPointsOfInterest();
35     }
36
37     /**
38     * Initializes the map with a fake map library.
39     */
40     initializeMap() {
41         this.map = new FakeMapLibrary(this.mapRef.current, {
42             center: this.state.center,
43             zoom: this.state.zoom,
44         });
45     }
46
47     /**
48     * Fetches routes from the fake API and sets it to the state.
49     */
50     fetchRoutes() {
51         fakeAPI.getRoutes().then(routes => {
52             this.setState({ routes });
53         });
54     }

```

```

55
56  /**
57   * Fetches points of interest from the fake API and sets it to the state.
58   */
59  fetchPointsOfInterest() {
60      fakeAPI.getPOIs().then(pointsOfInterest => {
61          this.setState({ pointsOfInterest });
62      });
63  }
64
65  /**
66   * Handles the change in zoom level.
67   * @param {number} zoom - The new zoom level.
68   */
69  handleZoomChange(zoom) {
70      this.setState({ zoom });
71  }
72
73  /**
74   * Handles the change in the center of the map.
75   * @param {number} lat - The latitude of the new center.
76   * @param {number} lng - The longitude of the new center.
77   */
78  handleCenterChange(lat, lng) {
79      this.setState({
80          center: { lat, lng }
81      });
82  }
83
84  /**
85   * Renders the routes on the map.
86   * @returns {React.Element[]} - An array of fake route components.
87   */
88  renderRoutes() {
89      return this.state.routes.map(route => (
90          <FakeRouteComponent key={route.id} data={route} />
91      ));
92  }
93
94  /**
95   * Renders the points of interest on the map.
96   * @returns {React.Element[]} - An array of fake point of interest components.
97   */
98  renderPointsOfInterest() {
99      return this.state.pointsOfInterest.map(poi => (
100          <FakePOIComponent key={poi.id} data={poi} />
101      ));
102  }
103
104  /**
105   * Renders the MapView component.
106   * @returns {React.Element} - The rendered component.
107   */
108  render() {
109      return (
110          <div ref={this.mapRef} className="map-view">
111              {this.renderRoutes()}
112              {this.renderPointsOfInterest()}

```

```

113         </div>
114     );
115 }
116 }
117
118 /**
119  * Represents a fake API for fetching data.
120  */
121 const fakeAPI = {
122     /**
123      * Fetches fake routes.
124      * @returns {Promise<Object[]>} - A promise that resolves to an array of route objects.
125      */
126     getRoutes: () => Promise.resolve([
127         { id: 1 },
128         { id: 2 }
129     ]),
130     /**
131      * Fetches fake points of interest.
132      * @returns {Promise<Object[]>} - A promise that resolves to an array of point of
133      * interest objects.
134      */
135     getPOIs: () => Promise.resolve([
136         { id: 'a' },
137         { id: 'b' }
138     ]);
139
140     /**
141      * Represents a fake map library for demonstration purposes.
142      */
143     class FakeMapLibrary {
144         /**
145          * Constructs the FakeMapLibrary.
146          * @param {HTMLElement} element - The HTML element to attach the map to.
147          * @param {Object} options - The map options.
148          */
149         constructor(element, options) {
150             // Simulate map initialization
151         }
152     }
153
154     /**
155      * Represents a fake route component for demonstration purposes.
156      * @extends React.Component
157      */
158     class FakeRouteComponent extends React.Component {
159         /**
160          * Renders the FakeRouteComponent.
161          * @returns {React.Element} - The rendered component.
162          */
163         render() {
164             return <div className="fake-route"></div>;
165         }
166     }
167
168     /**
169      * Represents a fake point of interest component for demonstration purposes.
170      * @extends React.Component
171      */
172     class FakePOIComponent extends React.Component {
173         /**
174          * Renders the FakePOIComponent.
175          */
176     }
177 }

```

```
170     * @returns {React.Element} - The rendered component.  
171     */  
172     render() {  
173         return <div className="fake-poi"></div>;  
174     }  
175 }  
176  
177 export default MapView;
```

Exercice p. 20 Solution n°5

Question 1

Pourquoi l'architecture feature-based est-elle considérée comme bénéfique pour les grandes équipes ?

- ☐ Elle propose une structure unique qui doit être suivie strictement
- ☒ Elle offre une lisibilité immédiate du code selon les besoins métier
- ☐ Elle facilite la création de couches techniques distinctes
-  Avec une architecture centrée sur les fonctionnalités, il est plus facile pour les développeurs de localiser et de comprendre le code relatif à une fonctionnalité spécifique, en particulier dans les grands projets où plusieurs développeurs travaillent en parallèle sur différentes fonctionnalités.

Question 2

Quelle couche dans une architecture layer-based est principalement responsable de la logique d'entreprise et du traitement des demandes venant de la couche de présentation ?

- ☐ Couche d'accès aux données
- ☐ Couche de présentation
- ☒ Couche métier
-  La couche métier, ou couche logique, contient la logique d'entreprise. C'est là que se prennent la plupart des décisions, que sont appliquées les règles métier, et que sont traitées les demandes venant de la couche de présentation.

Question 3

Après avoir installé le plugin eslint-plugin-react-native, où devriez-vous l'ajouter pour le configurer ?

- ☐ Dans un fichier appelé .eslintconfig
- ☐ Dans le fichier package.json
- ☒ Dans le fichier .eslintrc ou ses variantes (.eslintrc.js, .eslintrc.json)
-  Pour ajouter des règles spécifiques à React Native via le plugin, vous devriez le mentionner dans votre fichier de configuration ESLint, comme .eslintrc.

Question 4

Quelle règle ESLint vous avertirait si des styles sont définis, mais jamais utilisés dans un projet React Native ?

- ☒ "react-native/no-unused-styles"
- ☐ "react-native/no-raw-text"
- ☐ "react-native/no-color-literals"
-  La règle "react-native/no-unused-styles" prévient si des styles sont définis mais jamais utilisés.

Question 5

Quelle information le mot-clé @param de JSDoc décrit-il ?

- ☐ L'objectif général de la fonction
- ☒ Le paramètre d'une fonction
- ☐ La valeur renvoyée par la fonction
-  Le tag @param est utilisé pour décrire un paramètre de la fonction, son type, et éventuellement une description détaillée.

Question 6

Comment pouvez-vous générer une documentation HTML à partir des commentaires JSDoc dans vos fichiers JavaScript ?

- ☐ En utilisant la commande npm install jsdoc
- ☐ En ouvrant les fichiers dans un navigateur
- ☒ En exécutant la commande jsdoc . à la racine du projet
-  La commande jsdoc . indique à JSDoc de parcourir les fichiers JavaScript dans le répertoire courant et ses sous-répertoires pour générer la documentation.

Question 7

Après avoir exécuté la commande pour générer la documentation avec JSDoc, où est généralement stockée la documentation HTML ?

- ☒ Dans un dossier nommé out
- ☐ Dans un fichier documentation.html
- ☐ Dans un dossier nommé src
-  Par défaut, après avoir exécuté la commande JSDoc, la documentation HTML est créée dans un dossier généralement nommé out.